BASES DE DATOS · UNIDAD 3

# QATO'K

Modelo lógico de una plataforma de simulaciones profesionales, sobre una base de datos orientada a grafos

Neo4j como base principal · PostgreSQL como complementaria

EL PROBLEMA

# Hay una brecha entre estudiar y trabajar

Muchos egresados no saben si su experiencia académica basta, ni cómo demostrar lo que saben hacer.

**Para quién:** estudiantes de último semestre y recién egresados, con poco tiempo, que usan sobre todo el celular y cuidan su privacidad.

### Sus causas

- Formación predominantemente teórica
- Falta de experiencia laboral formal
- Pocas oportunidades de decidir con información incompleta
- Dificultad para comunicar competencias en entrevistas
- Ninguna retroalimentación sobre su desempeño

LA SOLUCIÓN

# Practicar el trabajo antes de tenerlo

Simulaciones cortas y repetibles donde cada decisión tiene consecuencias.

1. **Situación.** Un paso presenta un problema laboral: un correo, una reunión, un incidente.
2. **Decisión.** El usuario elige una opción, a veces con información incompleta.
3. **Consecuencia.** La opción genera resultados y lleva al siguiente paso.
4. **Retroalimentación.** Explica el acierto o el error y recomienda cómo mejorar.
5. **Progreso y evidencias.** Sube el nivel por habilidad y queda una evidencia que el usuario decide si comparte.

# Tres zonas unidas por Habilidad




**Contenido:** la simulación y sus pasos, opciones y consecuencias.

**Ejecución:** lo que hizo el usuario, decisión por decisión.

**Resultados:** su progreso y sus evidencias.

12 entidades · 17 relaciones · 14 tipos de relación

# Por qué una base de datos orientada a grafos

### El contenido ya es un grafo

Cada opción CONDUCE_A otro paso y varias rutas pueden converger: nodos y aristas dirigidas.

### Las preguntas son recorridos

Las 5 consultas del sistema cruzan de 4 a 8 entidades conectadas.

### Las relaciones llevan datos

DESARROLLA guarda peso y nivel esperado, sin tabla intermedia.

### Crece sin rediseñar

Un historial de progreso o una relación nueva se agregan como nodos y aristas.

# Una consulta, un recorrido

Consulta: ¿qué decidió el usuario, qué pasó y qué retroalimentación recibió?

```
MATCH (u:Usuario)-[:REALIZA]->(i:Intento)
MATCH (i)-[:REGISTRA]->(r:Respuesta)
MATCH (r)-[:CORRESPONDE_A]->(p:Paso)
MATCH (r)-[:SELECCIONA]->(o:Opcion)
OPTIONAL MATCH (o)-[:GENERA]->(c:Consecuencia)
OPTIONAL MATCH (r)-[:GENERA]->(f:Retroalimentacion)
RETURN r.secuencia, p.titulo, o.descripcion, c, f
```

**7**

entidades en un solo patrón

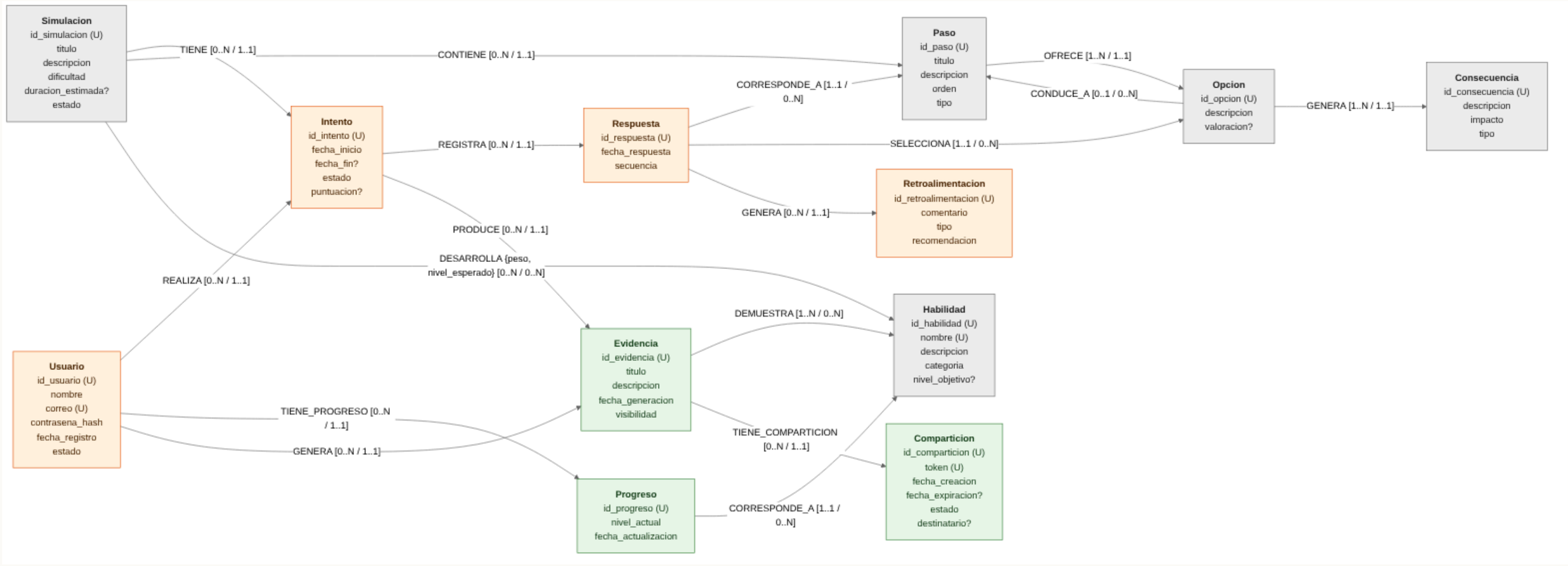
**6**

JOIN para lo mismo en SQL

# Los siete entregables

| # | Entregable | Cifra clave | Idea central |
|---|---|---|---|
| 1 | Requerimientos y consultas | 10 requerimientos · 5 consultas | Las consultas justifican el grafo |
| 2 | Diccionario de entidades | 12 entidades · 55 propiedades | Las claves foráneas son relaciones |
| 3 | Catálogo de relaciones | 17 relaciones · 14 tipos | Ninguna relación sobra |
| 4 | Restricciones de integridad | 25 restricciones · 5 categorías | Neo4j y la aplicación las garantizan |
| 5 | Cardinalidades | 3 entidades independientes | Ningún nodo queda huérfano |
| 6 | Diagrama del modelo lógico | 12 nodos · 17 relaciones | Todo el modelo en una vista |
| 7 | Conclusiones | 2 preguntas | Las relaciones definen el modelo |

# El modelo lógico completo




(U) = propiedad única · ? = opcional · [a / b] = cardinalidad: destinos por cada origen / orígenes por cada destino · gris: contenido · naranja: ejecución · verde: resultados

# Quién garantiza cada regla

### Neo4j, de forma nativa

- Identificadores únicos en las 12 etiquetas
- Correo, nombre de habilidad y token únicos
- Solo en la edición Enterprise: propiedades obligatorias y tipos de dato

### La aplicación, en cada transacción

- Rangos y valores permitidos (no hay CHECK)
- Relaciones obligatorias: sin nodos huérfanos
- Consistencia: la opción pertenece al paso respondido

**23/25**

restricciones necesitan validación en la aplicación

Las cardinalidades dicen qué crear junto: 3 entidades pueden existir solas y 9 nacen en la misma transacción que sus relaciones obligatorias.

NEO4J · PRUEBAS DE HUMO

# Lo esperado contra lo obtenido

Cada prueba es una consulta que devuelve **prueba**, **ok** y **detalle**. Si ok = false, la regla no se cumple.

| Grupo | Qué probamos | Esperado | Resultado |
|---|---|---|---|
| A · Esquema | Restricciones de unicidad e índices de consulta | 15 restricciones y 9 índices en línea | 15 y 9 en línea · ok 3/3 |
| B · Datos | Nodos por etiqueta y relaciones por patrón; los opcionales vacíos no se guardan | 51 nodos (12 etiquetas) y 71 relaciones (17 patrones) | 51 y 71 · ok 31/31 |
| C · Integridad | Cardinalidades y reglas de integridad: sin nodos huérfanos, rutas dentro de su simulación, rangos y catálogos | 0 nodos o valores que incumplan | 0 incumplimientos · ok 22/22 |
| D · Consultas | Las cinco consultas clave y la búsqueda por texto, con datos de ejemplo conocidos | Ana: «Tres entregas para mañana» primero; evolución 45 → 92 → 65 | Igual a lo esperado · ok 7/7 |
| E · Escrituras | Respuesta con un paso de otra simulación, MERGE repetido, intento en una simulación BORRADOR | No se crea nada indebido; el grafo sigue en 51 y 71 | 0 respuestas, 1 relación, 0 intentos · ok 5/5 |

**68 pruebas · 0 fallos**

Ejecutadas en Neo4j Desktop

# Decisiones de modelado clave

| Decisión | Qué se decidió | Por qué |
|---|---|---|
| Respuesta como nodo | Cada decisión es un nodo ligado a intento, paso, opción y retroalimentación | Una relación solo une dos nodos; la decisión toca cuatro |
| DESARROLLA con propiedades | Relación directa con peso y nivel esperado | Evita un nodo intermedio |
| Opción terminal | Una opción sin CONDUCE_A cierra el intento | El desenlace queda registrado en una respuesta |
| Toda opción con consecuencia | Opción → Consecuencia con cardinalidad 1..N | Cada decisión debe mostrar al menos un resultado |

Cada decisión se tomó comparando alternativas frente a las consultas que el sistema debe responder.

LO QUE SIGUE

# Limitaciones conocidas y pendientes

Son decisiones de diseño previstas para la siguiente etapa del proyecto.

- El desempeño se atribuye a cada habilidad por simulación, no por decisión individual
- La evolución de una habilidad se aproxima con los intentos; falta el historial de progreso
- Falta decidir qué datos van en PostgreSQL y cómo se sincroniza con Neo4j
- Falta definir cómo se calcula el nivel de cada habilidad

CONCLUSIÓN

# Las entidades son el vocabulario; las relaciones y sus reglas son la gramática.

Las preguntas importantes de QATO’K son recorridos, y un grafo está hecho para recorrer.

**Gracias · ¿Preguntas?**